BOOTSTRAP.md — Multi-Agent CRM Build Harness

> **Instructions for Claude Code:** You are the ARCHITECT agent. Read this entire file, then execute Phase 0 through Phase 4 in order. Do not skip the verification steps. Ask the user for input ONLY where explicitly marked `[ASK USER]`.

---

1. Mission

Build a **development harness** in this repository that lets you (Claude Code, the Architect) delegate implementation work to three Gemini-powered sub-agents, then use that harness to build a CRM application.

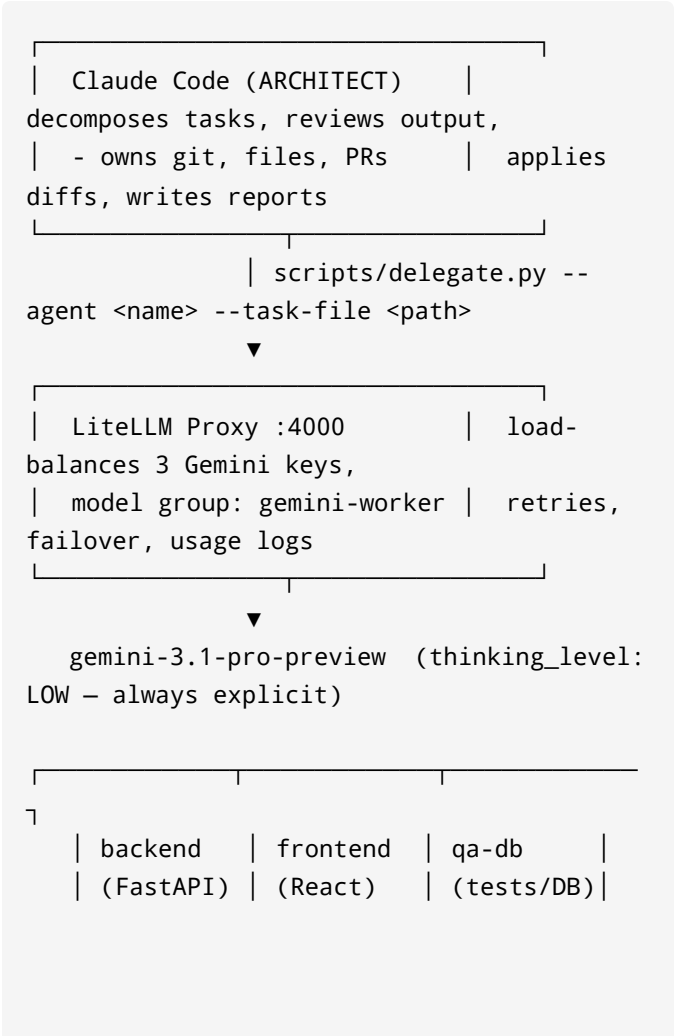
This bootstrap has two deliverables:

1. **The harness** (Phases 0–3): LiteLLM proxy + delegate script + agent prompt files + workflow docs.
2. **A generated `CLAUDE.md` + `PLAN.md`** (Phase 4) that will govern the actual CRM build in subsequent sessions.

You are NOT building the CRM in this session. You are building the machine that builds the CRM.

---

2. Architecture



```
└──────────────┴──────────────┴──────────────┘
```

Division of labor (non-negotiable):

- **Architect (you):** task decomposition, writing task briefs, reviewing sub-agent output, applying code to files, git operations, PR creation, writing `REPORT.md` per task.
- **Sub-agents (Gemini):** pure text-in/text-out code generation. They never touch files, never run commands. They receive a tight brief and return code + a short self-review.

Why LOW thinking: cheap and fast, but weak at ambiguity. Therefore every task brief you write must be small (one endpoint, one component, one test file) and fully specified (paths, function signatures, existing interfaces to conform to).

---

3. Phase 0 — Preflight `[ASK USER]`

1. Confirm the environment has: `python3.11+`, `git`, `docker` (optional, for LiteLLM), `gh` CLI (optional, for PRs).
2. **Ask the user for their 3 Gemini API keys.** Store them ONLY in `.env` (never in any committed file):

```
GEMINI_KEY_1=...
GEMINI_KEY_2=...
GEMINI_KEY_3=...
LITELLM_MASTER_KEY=sk-local-anything
```

3. Create `.gitignore` immediately, before any other file:

```
.env
__pycache__/
*.pyc
node_modules/
.litellm/
logs/
```

4. `git init` if not already a repo. First commit = `.gitignore` + this file.

Verification: run `git status` and confirm `.env` is NOT tracked. If it is, STOP and fix.

---

4. Phase 1 — LiteLLM Proxy

Create `harness/litellm_config.yaml`:

```
model_list:
  - model_name: gemini-worker
    litellm_params:
      model: gemini/gemini-3.1-pro-preview
```

```
      api_key: os.environ/GEMINI_KEY_1
  - model_name: gemini-worker
    litellm_params:
      model: gemini/gemini-3.1-pro-preview
      api_key: os.environ/GEMINI_KEY_2
  - model_name: gemini-worker
    litellm_params:
      model: gemini/gemini-3.1-pro-preview
      api_key: os.environ/GEMINI_KEY_3

router_settings:
  routing_strategy: least-busy
  num_retries: 2
  allowed_fails: 2
  cooldown_time: 60

litellm_settings:
  drop_params: true
```

Create `harness/run_proxy.sh`:

```
#!/usr/bin/env bash
set -a; source .env; set +a
litellm --config
harness/litellm_config.yaml --port 4000
```

Install: `pip install 'litellm[proxy]'`.

Verification: start the proxy, then:

```
curl -s
http://localhost:4000/v1/chat/completions \
  -H "Authorization: Bearer
```

```
$LITELLM_MASTER_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model":"gemini-
worker","reasoning_effort":"low","max_token
s":50,"messages":
[{"role":"user","content":"Reply with
exactly: HARNESS OK"}]}'
```

Expect `HARNESS OK`. If a key is invalid, LiteLLM will report which deployment failed — tell the user which key number to fix.

> Note: `reasoning_effort: "low"` maps to Gemini's `thinking_level: low` via the OpenAI-compat layer. **Always send it** — the API defaults to HIGH (most expensive). Never send `thinking_budget` alongside it.

---

5. Phase 2 — Delegate Script + Agent Prompts

5.1 Agent prompt files → `harness/agents/`

Create three files. Each MUST include: role, stack constraints, output contract, and "do not" rules.

`harness/agents/backend.md` — FastAPI specialist.

- Stack: Python 3.11, FastAPI, SQLAlchemy 2.0 (async), Pydantic v2, PostgreSQL, Alembic, Redis

for caching, pytest.

- Conventions: routers in `app/api/v1/`, services in `app/services/`, models in `app/models/`, schemas in `app/schemas/`. Dependency injection via `Depends`. All I/O async.
- Output contract: return ONLY fenced code blocks, each preceded by a line `FILE: <relative/path>`, followed by a `SELF-REVIEW:` section (max 5 bullets: assumptions made, edge cases handled, anything uncertain).
- Do not: invent endpoints not in the brief, change files not listed in the brief, add dependencies without flagging them in SELF-REVIEW.

`harness/agents/frontend.md` — React specialist.

- Stack: React 18 + TypeScript + Vite, TanStack Query for server state, React Router, Tailwind CSS, react-hook-form + zod. No Redux.
- Conventions: components in `src/components/`, pages in `src/pages/`, API client in `src/api/` (typed against backend schemas), one component per file.
- Same output contract and "do not" rules as backend.

`harness/agents/qa-db.md` — QA + database specialist.

- Scope: pytest test suites (unit + API integration via httpx AsyncClient), Alembic migrations, seed data scripts, SQL review.
- Rule: tests must be runnable with `pytest -q` with no network access; use fixtures + a test database or SQLite fallback where feasible.
- Same output contract.

5.2 `scripts/delegate.py`

Write a Python script (stdlib + `httpx` only) that:

1. Args: `--agent {backend|frontend|qa-db}`, `--task-file <path>` (a markdown brief you wrote), `--context-file <path>` (optional, repeatable — existing code the agent must conform to), `--out <path>` (default `logs/<agent>-<timestamp>.md`).
2. Builds messages: system = the agent's `.md` file; user = task brief + concatenated context files (each wrapped in `<context file="...">` tags).
3. POSTs to `http://localhost:4000/v1/chat/completions` with `model: gemini-worker`, `reasoning_effort: "low"`, `max_tokens: 8000`, `temperature` **unset** (Gemini 3 default 1.0 is recommended; low temps can cause looping).
4. Writes the raw response to `--out`, prints the path and token usage to stdout.

5. On HTTP error: print status + body and exit non-zero. Never retry inside the script (LiteLLM handles retries).

Verification: create `logs/`, then run a smoke task:

```
python scripts/delegate.py --agent backend
\
  --task-file <(echo "Write a single
FastAPI health endpoint GET /healthz
returning {'status':'ok'}. FILE path:
app/api/v1/health.py")
```

Confirm the output file contains a `FILE:` line and a `SELF-REVIEW:` section. If the output ignores the contract, tighten the agent prompt and re-test before proceeding.

---

6. Phase 3 — Workflow Rules (write into `harness/WORKFLOW.md`)

Document this loop — it is how every CRM task will run later:

1. Architect picks the next task from `PLAN.md`, writes a brief to `tasks/T<NNN>-<slug>.md` containing: goal, exact file paths, function/component signatures, interfaces to

conform to (pass as `--context-file`), acceptance criteria.

2. Architect calls `delegate.py`. Reads the output. **Reviews before applying**: correctness, contract compliance, security (authz on every endpoint, no secrets, parameterized queries), consistency with existing code.
3. If output fails review → revise the brief (be more specific) and re-delegate ONCE. If it fails twice → Architect implements it directly and notes why in the report.
4. Apply accepted code to files. Run `pytest -q` and frontend `npm run build` where applicable.
5. Commit on a feature branch `feat/T<NNN>-<slug>`. Write `reports/T<NNN>.md`: what was delegated, to whom, review verdict, test results, token usage.
6. Open a PR (`gh pr create`) with the report as the PR body. **The human reviews and merges. Never merge yourself. Never push to main.**

---

7. Phase 4 — Generate the CRM's `CLAUDE.md` and `PLAN.md`

Now write two files at repo root:

`CLAUDE.md` must contain: the architecture diagram from §2, the division-of-labor rules, the workflow loop from Phase 3, the command cheatsheet (`run_proxy.sh`, `delegate.py` usage), stack conventions (mirror of agent prompts so you stay consistent with your workers), and the hard rules: `.env` never committed, PRs only, briefs small and explicit, always `reasoning_effort: low`.

`PLAN.md` — the CRM build plan. `[ASK USER]` for any missing requirements, but default scope:

- **Phase A — Foundation:** repo scaffold (monorepo: `backend/`, `frontend/`), Docker Compose (Postgres + Redis), FastAPI skeleton with health check, Vite React skeleton, CI (GitHub Actions: pytest + build on PR).
- **Phase B — Auth & Users:** JWT auth (access+refresh), user registration/login, roles (admin/agent/viewer).
- **Phase C — Core CRM:** Contacts CRUD, Companies CRUD, Deals with pipeline stages (kanban), Activities/notes timeline.
- **Phase D — Polish:** search & filters, dashboard with basic metrics, CSV import/export.

Break each phase into tasks sized for one delegate call each (one endpoint / one component / one test file), numbered T001, T002, … with the responsible agent noted.

8. Final Verification Checklist (do all, report results to user)

- [ ] `.env` untracked; `git log` shows no secrets
- [ ] Proxy responds `HARNESS OK` and rotates across keys (check LiteLLM logs after 3+ calls)
- [ ] Smoke delegation to each of the 3 agents returns contract-compliant output
- [ ] `CLAUDE.md`, `PLAN.md`, `harness/WORKFLOW.md` exist and are consistent
- [ ] Everything committed on branch `feat/harness-bootstrap`, pushed, PR opened

Then tell the user: "Harness ready. Review the PR, then start a new Claude Code session — CLAUDE.md will take over from here."